

Parallel protein secondary structure prediction schemes using Pthread and OpenMP over hyper-threading technology

Wei Zhong · Gulsah Altun · Xinmin Tian ·
Robert Harrison · Phang C. Tai · Yi Pan

Published online: 24 February 2007
© Springer Science+Business Media, LLC 2007

Abstract Protein secondary structure prediction has a fundamental influence on today's bioinformatics research. In this work, tertiary classifiers for the protein secondary structure prediction are implemented on Denoeux Belief Neural Network (DBNN) architecture. Hydrophobicity matrix, orthogonal matrix, BLOSUM62 matrix and PSSM matrix are experimented separately as the encoding schemes for DBNN. Hydrophobicity matrix, BLOSUM62 matrix and PSSM matrix are applied to DBNN architecture for the first time. The experimental results contribute to the design of new encoding schemes. Our accuracy of the tertiary classifier with PSSM encoding scheme reaches 72.01%, which is almost 10% better than the previous results obtained in 2003. Due to the time consuming task of training the neural networks, Pthread and OpenMP are employed to parallelize DBNN in the Hyper-Threading enabled Intel architecture. Speedup for 16 Pthreads is 4.9 and speedup for 16 OpenMP threads is 4 in the 4 processors shared memory architecture. Both speedup performance of OpenMP and Pthread is superior to that of other research. With the new parallel training algorithm, thousands of amino acids can be processed in reasonable amount of time. Our research also shows that Hyper-Threading technology for Intel architecture is efficient for parallel biological algorithms.

W. Zhong
Division of Math and Computer Science, University of South Carolina Upstate, Spartanburg,
SC 29303, USA
e-mail: wzhong@uscupstate.edu

G. Altun · R. Harrison · Y. Pan (✉)
Department of Computer Science, Georgia State University, Atlanta, GA 30303, USA
e-mail: pan@cs.gsu.edu

X. Tian
Software Solution Group, Intel Corporation, Santa Clara, CA, USA

R. Harrison · P.C. Tai
Department of Biology, Georgia State University, Atlanta, GA 30303, USA

Keywords Neural networks · Protein secondary structure prediction · Parallel architecture · Speedup · DBNN (Denoeux Belief Neural Network) · MPI (Message Passing Interface) · OpenMP · Pthread · Hyper-threading · BLOSUM62 matrix · Hydrophobicity matrix · PSSM (Position Specific Scoring Matrix)

1 Introduction

Protein tertiary structure plays a very important role in determining its possible functional sites and chemical interaction with other related proteins. Prior knowledge about protein three-dimensional structure is very helpful for protein engineering and drug design. For example, if the structure of a certain protein that causes a disease is determined, a chemical reaction related to this protein can be found out to facilitate drug research. Researchers try to determine the tertiary structure of proteins using X-ray crystallography and Nuclear Magnetic Resonance (NMR). Both methods are time consuming and expensive. Sometimes X-ray crystallography and NMR fail to find out the three-dimensional coordinates of an amino acid. As a result, the gap between protein sequence and its structure has widened substantially due to the high throughput sequencing techniques. The growing gap increases the significance of predicting the protein tertiary structure. If the secondary structure of a protein is available, it is possible to infer a comparatively small number of possible tertiary structures. In other words, prediction of protein secondary structure is an intermediary step to explore its tertiary structure. Many biochemical tests suggest that a sequence determines conformation completely because all the information, which is necessary to specify protein interaction sites with other molecules, is embedded into its amino acid sequence. This close relationship between a sequence and a structure forms the theoretical basis for protein structure prediction. In this work, four encoding schemes have been experimented for the tertiary classifier of Denoeux Belief Neural Network (DBNN). In order to speed up the training process, DBNN is parallelized with Pthread and OpenMP separately and their experimental results are compared. Furthermore, Hyper-Threading technology enables us to examine high performance measurements in terms of program execution time and speedup.

In Sect. 2, the background information on protein secondary structure prediction is introduced. In Sect. 3, the neural network architecture used in this work is explained. In Sect. 4 and Sect. 5, training data, testing methods and the encoding schemes is given. In Sect. 6, detailed descriptions of the parallelization methods and the Hyper-Threading technology is explained. The experimental results are presented in Sect. 7. Finally, the conclusion and future work is given in Sect. 8.

2 Protein secondary structure prediction

A protein has four levels of structural hierarchy, which are primary, secondary, tertiary, and quaternary. Primary structure of polypeptide defines the linear sequence of amino acid that makes up the chain. Secondary structure of protein describes three main conformations of the polypeptide. Secondary structure has three main conformation including alpha Helix (H), Beta sheet (B) and Coil (C). Alpha helix is

the chain of the polypeptide in the shape of a cylindrical and spiral. Beta sheet includes a couple of segments lying side by side. The rest of protein chains are called coils, including turns and loops. These turns and loops are usually located in the surface of proteins and serve as the active interaction sites between proteins and other molecules. Tertiary structure constitutes diverse side chains stabilized by noncovalent bonds [12].

During the training session, the predicted structure result is compared with available structure information of the sequence in order to evaluate the amount of error generated by the neural network. The prediction model tries to associate the pattern of input with its output. After training, the prediction model can predict the structure of a given sequence based on knowledge of the previous input patterns. Q3 is used throughout this paper to compare the prediction accuracy among different research.

3 Denoex Belief Neural Network (DBNN)

Some research efforts have been taken to carry out protein secondary structure prediction, using the neural network. Qian and Sejnowski [17] predict structure of globular proteins based on a non-linear neural network to reach 64.3% accuracy. Rost and Sander [20] predict protein secondary structure using two-layered feed forward neural network with better than 70% accuracy. In Rost's [20] research, evolutionary information is incorporated into the input profile in the form of multiple sequence alignment to increase prediction accuracy by 6%. In this research, DBNN is used to perform protein secondary structure prediction. DBNN is a multilayer neural network with one input layer, two hidden layers and one output layer and is based on the Dempster-Shafer theory [6]. As shown in the Eq. (1), the similarity of the input samples to each prototype is determined by Euclidean metric, which is measured by the sum of squared differences between samples and prototypes. The nonlinear activation function of the Eq. (2) and the Eq. (3) gives Basic Belief Assignments (BBAs) [6]. BBAs from each prototype are combined to generate final class membership for input samples with the Dempster's rule of combination according to the Eq. (4) [6]. In the output layer, the class with the maximum strength of belief is assigned to input samples. This step is illustrated in the Eq. (5). In order to optimize the values for prototypes, α and γ , the complex gradient function is used in order to search local minimum of the error function through traditional backpropagation algorithm [2]. In this backpropagation algorithm, the parameter gradients can be calculated based on the derivatives for prototypes, α and γ and the amount of error in each iteration [2]. At the end of each iteration, the parameter values for prototypes, α and γ can be reevaluated according to error gradients, parameter gradients and the learning rate. Learning rate is adjusted by Silva-Almeida method [22]. Usually after 800 iterations, the convergence criterion is met and training process is terminated. During initialization of prototypes, the training samples are clustered into several similar groups with k-means clustering algorithm [2]. Each group represents one prototype to increase the diversity of the prototype. Since the backpropagation function is very complex, its details will not shown in this paper. The following equations are produced based on the Dempster-Shafer Theory [6].

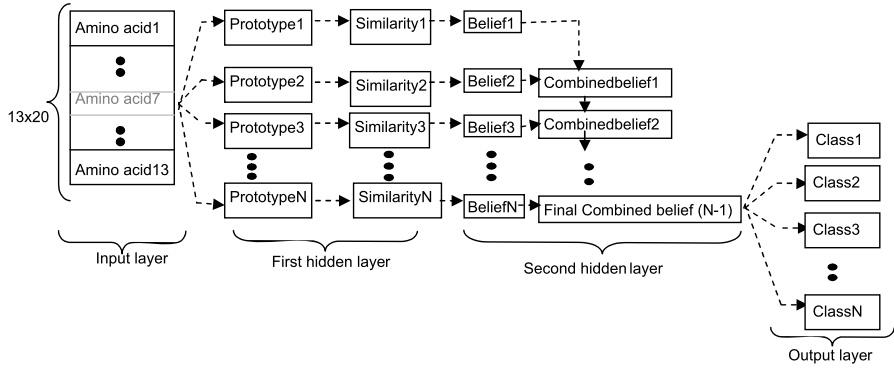


Fig. 1 DBNN architecture

First hidden layer of DBNN network uses the following equation:

$$\text{activate}[k] = \exp(-\gamma[k]^2 \times \text{similarity}[k]^2) \times \alpha_p(k) \quad (1)$$

$$\alpha_p(k) = \frac{1}{1 + \exp(-\alpha[k])} \quad (2)$$

$$B[k] = \text{activate}[k] \times \text{membership}[k] \quad (3)$$

Second hidden layer uses the following equation:

$$CB[l] = CB[l](B[k \times cn + l] + B[k \times cn + cn] + B[k \times cn] \times CB[cn]) \quad (4)$$

Output layer uses the following equation:

$$CB[l]_{\text{MAX}} \quad (5)$$

where CB is combined belief assignment, B is belief of each prototype and cn is the class number.

Structure of neural network is based on Dempster-Shafer Theory [6]. Multilayer architecture of DBNN is illustrated in Fig. 1.

Arjunan [2] adopts the orthogonal matrix as the encoding scheme and tests RS126 data set to obtain 62.39% prediction accuracy for the tertiary classifier of DBNN architecture.

4 Training data and test methods

Cuff and Barton [5] proposed the CB513 data set for the protein secondary structure prediction. We used the CB513 data set in our experiments. CB513 contains 513 non-homologous proteins with about 80,000 amino acids. Seven-fold cross validation is used to test prediction accuracy in the [10, 13, 18]. In the seven-fold cross validation test, the data set is divided into seven segments. Six segments are used for training

and one segment is used for testing. This process is repeated for seven times until all seven segments have been chosen as the testing set.

The sliding window method is used for the testing and training process in our tests. Each sliding window with thirteen successive residues represents one input profile to predict the secondary structure of the residue at the center of the window. The performance of different window sizes in our study shows a window of thirteen is reasonable since smaller window sizes may lose vital information about local interactions among neighboring residues and bigger windows may introduce high signal to noise ratio. Input layers have $(20 \text{ binary bits per residue}) \times (13 \text{ residues}) = 260 \text{ nodes}$. The network has one activation-hidden layer and one combination hidden layer. The output layer has three nodes for helix, sheet and coil. The final output structure is derived from the maximum value of these three nodes.

DSSP [19] are one of the popular methods used to decide the secondary structure from the experimentally determined tertiary structure. The DSSP initially assigns the secondary structure to eight different classes. Before going through a training process, the structure is converted to three classes based on the following method: H, G and I to H; B and E to E; all others to C. After converting eight classes to three classes, Q_3 is used in order to measure the prediction accuracy. Q_3 is three-state per residue overall percentage of correctly predicted residues [19].

$$P_T = P_{\text{helix}} + P_{\text{sheet}} + P_{\text{coil}}$$

R_T = Total number of residues

$$Q_3 = \frac{P_T}{R_T} \times 100\%$$

P_{helix} , P_{sheet} and P_{coil} are number of residues correctly predicted in these three classes, respectively [2].

5 Encoding scheme

The orthogonal matrix, hydrophobicity matrix, BLOSUM62 matrix and PSSM matrix are used for the DBNN's input profile. Orthogonal matrix encoding assigns a unique binary vector to each twenty distinct residues. The unique binary vector is (1,0,0...), (0,1,0...), (0,0,1...) [10]. Orthogonal encoding ensures nonoverlapping information for twenty amino acids. The hydrophobicity matrix considers physico-chemical properties, which may affect conformation of a polypeptide chain. Strongly polarized bonds are very important to determine the reactivity of molecules [12]. Nonpolar molecules are relatively inert [12]. Polarity can also influence conformation of protein chains. Nonpolar groups tend to gather within the interior of most soluble proteins and minimize their exposure to polar surroundings [12].

Structure is conserved more than sequence. This principle forms the theoretical basis for BLOSUM62 matrix. The amino acid substitution pattern from approximately 2000 conserved amino acid blocks are closely observed [11]. The results obtained from these observations produce log-odds scores to show possibility that the given amino acid pairs will replace each other. In the BLOSUM62 matrix, a pair of amino

acids with similar chemical properties is more likely to replace each other while preserving structural properties and is given a positive score in the matrix. Conversely, the given amino acid pairs with very different chemical and physical properties are given a negative score.

PSI-BLAST produces a PSSM from a multiple alignment. PSSM gives high scores to conserved position and zero scores to weakly conserved positions [1]. Because the PSSM matrix gives different substitution patterns at different positions of protein sequences, the PSSM matrix is much more sensitive to discover distantly related protein sequences.

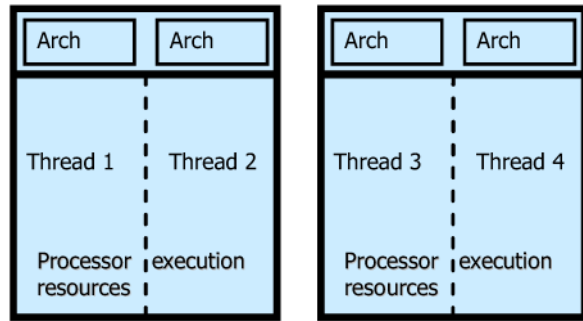
6 Parallelization

Training neural networks with the batch mode is a very slow and time consuming task because a large data set of thousands of amino acids and different encoding schemes has to be attempted for many times. However, the natural characteristics of a neural network allow itself to be easily parallelized because of its pipelined communication pattern and simple processing unit. Once parallelism is incorporated into the neural network, significant amounts of training time can be saved.

Data partitioning and task partitioning are two important parallelization techniques for backpropagation neural networks. In data partitioning parallelism, each processor with the same copy of the neural network parameters works on one portion of the training data. After one iteration, the results from all processors are accumulated to update network parameter [25]. Data partitioning is feasible for SIMD shared memory architecture. Weishaupl and Schikuta [26] adopted a data parallel approach for cellular neural networks to perform image-processing task. Fedorova and Terekhoff [8] implemented data parallelization for medium-size feed-forward neural networks, using the Message Passing Interface (MPI) library.

In task partitioning parallelism, tasks are partitioned among the processors based on the network architecture. Liu and Wilcox implemented a large backpropagation neural network to predict protein tertiary structure by task partitioning parallelism [15]. In their application, input nodes, output nodes and hidden nodes are distributed among processors evenly [15]. The linear relationship between the computation time and the number of processors has been observed [15]. Fathy and Syiam [9] implemented task partitioning for backpropagation neural networks. In their approach, each layer of the neural network is divided into four groups of neurons and each group is assigned to one processor in each stage of neural network [9]. Suresh and Omkar [23] also used task-partitioning parallelism for the memory neuron network using MPI library. For task partitioning parallelism, the speedup increases with a larger size of a network for a given size of training set because each processor can perform larger amount of computation before communicating with other processors to exchange the weight updates [9]. In other words, task-partitioning parallelism will be effective only when the amount of computation and network parameters is high [23]. Parallel DBNN network has been implemented by Arjunan [2] with MPI and a speedup of 3.87 has been reached for four processors. Since our neural network is small and the number of processors is fixed in our approach, data-partitioning parallelism is more suitable to our application.

Fig. 2 Two physical processors and four logical processors in courtesy of [16]



6.1 Hyper-Threading Technology

The Simultaneous Multithreading (SMT) is a method that allows multiple threads to issue instructions in each cycle. SMT maximizes performance and power consumption of the CPU. It has been identified as one of the best parallel multithreading techniques among the thread level parallelism techniques [7]. Hyper-Threading had developed the SMT for the Intel architecture on the Intel^(R) XeonTM [16]. In the Hyper-Threading enabled architecture, a single processor can be divided into multiple logical processors when needed. These logical processors can execute the instructions simultaneously. While each logical processor shares the physical execution resources efficiently, it keeps its own copy of the architecture state. Therefore, Hyper-Threading gives two virtual processors out of one physical processor. Each logical processor performs at approximately 60–70% of the capacity of one physical processor [16]. Two physical processors with Hyper-Threading technology are shown in Fig. 2. Programs must be parallelized and be executed in multiple threads in order to obtain the performance gains that Hyper-Threading Technology brings. Hyper-Threading Technology can be applied both data partitioning parallelism and task partitioning.

6.2 Pthread and OpenMP

Multiple threads bring parallelism for sequential programs. Thread usage is based on shared memory. Two popular parallelization methods used on shared memory are POSIX threads (Pthreads) and OpenMP. Pthreads are a very popular API for threading an application [3]. OpenMP API is a multi-platform shared-memory parallel programming, which supports C/C++ and FORTRAN [4].

Parallelizing a sequential program with OpenMP is much easier than that with Pthread because when Pthreads are used, the programmer has to deal with low-level details of thread creation, management and synchronization. Even though OpenMP is generally more suitable for data parallelization, this principle may not be applied to some applications. Therefore, we still want to compare the performance of Pthread and OpenMP in this study.

In our project, these two different parallelization methods are used separately on the same neural network and the performance for two parallelization methods are

compared. Hyper-Threading Technology enabled architecture is the test bed for both methods. The performance results are very good when Hyper-Threading is used. We present our experimental results in detail in Sect. 7.

Other researchers have used OpenMP and MPI for paralleling neural networks. Johansson and Lansner [14] have implemented a parallel Bayesian Neural Network with Hypercolumns using OpenMP and MPI. It is shown that OpenMP is a good alternative for a medium sized Bayesian Confidence Propagation Neural Network (BCPNN) while MPI is an alternative for a large number of processors [14]. The problem size has to increase substantially when the number of processors goes up in order to keep linear speed up when MPI is used [24].

6.3 Programming environment and implementation details

An Intel® OpenMP C++/Fortran compiler for Hyper-Threading technology is used to test Pthreads and OpenMP performance in our experiments. This compiler has advanced optimization techniques for the Intel processor [21]. Speedup and program execution time for Pthreads and OpenMP are measured.

The “poweredge6600 server” with four processors from Dell is used in this study. Because of the Hyper-Threading technology, it behaves like eight logical processors. Eight or more threads are used as shown in Fig. 3. The server architecture is optimized for four Intel Xeon processor symmetric multi-processing (SMP). The operating system is Linux.

In order to show how our parallel algorithm works, an example for five threads is illustrated in Fig. 4 when Pthread is used. One of the threads is called the master thread and the others are referred to as slave threads. The parallelization algorithm is explained gradually below. The whole data file has been divided into several sections. Each thread is assigned to one of these sections.

After one master and four slave threads are created, the master thread enters a waiting state. Then the following steps are followed:

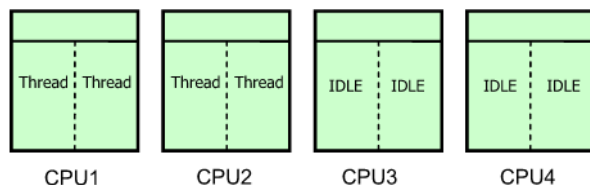
Step 1: The slave threads read network parameters from the shared memory. In this case, every thread has the same copy of the neural network.

Step 2: Each slave thread gets its portion of the training set. The training set is divided equally among all threads to establish a balanced workload.

Step 3: After calculating the errors, each slave thread updates its private memory space allocated for it in the shared memory. Every time a slave thread updates the memory space, it enters the waiting state.

Step 4: The last thread that updates the private memory space signals the main thread to wake up. After that, the slave thread enters the waiting state as well. Now, all slave threads are in the waiting state and doing nothing.

Fig. 3 Four physical processors behaving like eight logical processors



Step 5: The master thread wakes up upon receiving the wake up signal and reads the errors from the private memory space allocated for slave threads.

Step 6: The master thread updates the weight coefficients of the network.

Step 7: The master thread sends a broadcast signal to wake up all the slave threads. Upon sending this signal, the master thread enters the waiting state. The slave threads start the process from Step 1 again and the cycle goes on.

An example of the program codes for Pthread and OpenMP implementations are given in Fig. 5 and Fig. 6. The OpenMP has a much shorter code than Pthread because OpenMP hides the users from the low-level details of iteration space partitioning, data sharing, and thread scheduling and synchronization. As a result, one simple command block can be used to create and synchronize different child threads under control of one master thread.

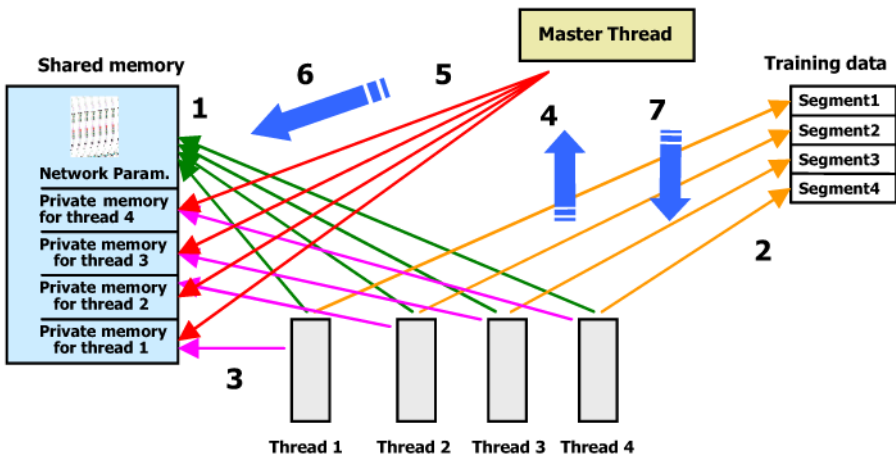


Fig. 4 Implementation details of five Pthreads

Child Thread

```
calculate(protos, beta, gamma, alpha, filearray[tid], tid);
pthread_mutex_lock(&count_mutex_cond2);
count=count+1; /* signaling main thread*/
if (count == COUNT_LIMIT)
pthread_cond_signal(&count_threshold_cvp)
thread_cond_wait(&count_threshold_cond2, &count_mutex_cond2);
```

Parent Thread

```
pthread_cond_wait(&count_threshold_cv, &count_mutex_cond2);
calculateGradient(prevDprotos, prevDbeta, prevDgamma, prevDalpha, &ePrev);
pthread_cond_broadcast(&count_threshold_cond2);
```

Fig. 5 Pthread code

```
omp_set_num_threads(NUM_THREADS);
#pragma omp parallel private(nthreads, tid)
{
    tid = omp_get_thread_num(); /* Obtain and print thread id */
    calculate(protos, beta, gamma, alpha, filearray[tid], tid);
}
Neural-network-update-function( );
```

Fig. 6 OpenMP code

7 Experimental results

In this section, prediction accuracy for tertiary classifiers of four encoding schemes is compared. Furthermore, the performance of Pthread and OpenMP is presented.

7.1 Test accuracy for secondary structure prediction

In Fig. 7, hydro represents the hydrophobicity matrix, orthogonal represents the orthogonal matrix, BLOSUM represents the BLOSUM62 matrix and PSSM represents the PSSM matrix. Among four encoding schemes, the accuracy of the hydrophobicity matrix for the tertiary classifier is much worse than that of the BLOSUM62 matrix in Fig. 7. The poor prediction accuracy may indicate that the level of polarity for amino acids alone may not play decisive role in determining the conformation of the peptide chain since many other chemical and physical properties may also affect formation of protein structure. The prediction accuracy of the BLOSUM62 matrix is much higher than that of hydrophobicity matrix since BLOSUM62 matrix is obtained from observation of the different types of amino acids substitution after alignment with the most common blocks from 500 protein families. The value of BLOSUM62 matrix may indicate an evolutionary relationship between different protein families in a general way. Prediction accuracies from the BLOSUM62 matrix and orthogonal matrix are almost the same, showing that the orthogonal matrix can represent information of amino acids as sufficient as the BLOSUM62 matrix. The result of the PSSM matrix is the best among four encoding schemes. Since the PSSM matrix shows specific amino acid substitution patterns for one protein family and the BLOSUM62 matrix indicates averaging substitution patterns for all protein families, the PSSM matrix outperforms BLOSUM62 matrix. Our tertiary classifier improves Arjunan's [2] work by 10%.

Fig. 7 DBNN tertiary classifier

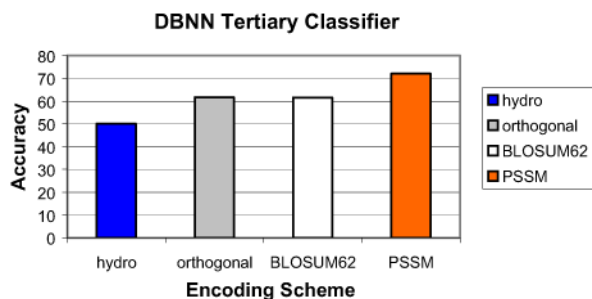


Fig. 8 Execution times for Pthread and OpenMP

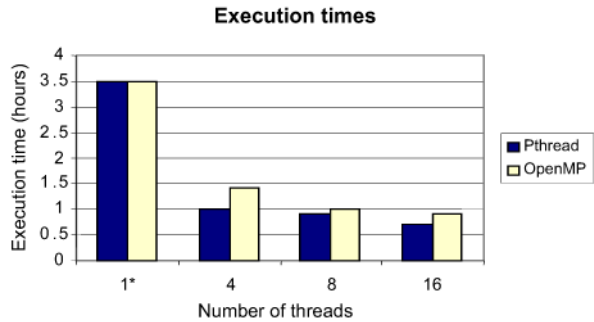
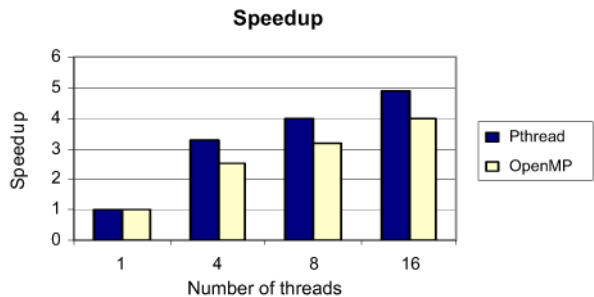


Fig. 9 Speedup values for Pthread OpenMP



7.2 Comparing Pthread and OpenMP implementations

The total program execution time (in hours) for Pthread and OpenMP when different numbers of threads are used is shown in Fig. 8 and the speedup values for Pthread and OpenMP is shown in Fig. 9.

The compiler has built-in optimizations specific to Intel's Hyper-Threading architecture. It also integrates parallelization tightly with other advanced optimization techniques to achieve better cache locality and reduce the overhead of data sharing among threads [21]. The execution times of OpenMP and Pthread differ as well. As can be seen from Fig. 8, Pthread gives a lower execution time than OpenMP. The speedup values for Pthread and OpenMP are presented in Fig. 8. Pthread gives a higher speedup value than that of OpenMP. This higher speedup ratio results from our neural network program implementation. When Pthread is used for parallelizing, the threads can be created only once and used many times with explicit synchronization among threads. However, when OpenMP is used for parallelization, the parallel region is created within the local function and the local function is called many times. Consequently, there is no way to keep the threads alive once the local function is returned and memory space allocated to local function is reclaimed. Therefore, all threads are destroyed after the return of the local function. Therefore, OpenMP loses performance efficiency by creating and destroying threads every time the local function is called. On the other hand, the same threads can be used repeatedly once threads are created in the Pthread implementation. Various coding techniques for the OpenMP program have been used to create threads outside the local function so that threads can stay alive during the execution of the program. However, these techniques

only produce inconsistent results. This was one of the drawbacks in the OpenMP program. Although there is some communication overhead for the threads to signal each other in the Pthread implementation, this communication overhead is much less than the overhead produced by the thread creation and destruction of OpenMP. The advantage of the Pthread program produces a better speedup value.

When the number of threads continues to grow, the increasing cost of context switching and synchronization among the threads will decrease the efficiency of OpenMP and Pthread. As a result, the speedup will go down eventually after the number of threads becomes very big.

While Arjunan [2] uses MPI to parallelize DBNN and reaches the speedup of 3.87 for four processors, the speedup for 16 Pthreads is 4.9 and the speedup for 16 OpenMP threads is 4. The parallel performance of OpenMP and Pthread is superior to that of MPI.

8 Conclusions and future work

Protein structure prediction is one of the crucial and imminent problems for bioinformatics. In this paper, the tertiary classifiers for DBNN are implemented to predict the protein secondary structure with four encoding schemes. In particular, the hydrophobicity matrix, the BLOSUM62 matrix and the PSSM matrix are tried for the first time for DBNN. The results analysis for these four encoding schemes provide important clues for designing more advanced encoding schemes. The combined input profile of PSSM and the frequency matrix can be used to further improve the test accuracy. The results indicate the tertiary classifier with the PSSM encoding performs 10% better than that of previous research [2]. To speed up the training process, DBNN is also parallelized with Pthread and OpenMP. Higher speedup and lower execution time are reached when Pthread is used. Hyper-Threading technology is effective for the biological parallel algorithms and will be beneficial for future parallelization research. The parallel training program can make it possible to process thousands of amino acids in a short amount of time in order to speed up tedious and intensive computational biomedical jobs. Currently, all the prediction systems only consider local interaction within 13–20 consecutive residues. This may be one of the major drawbacks of the current prediction method since the interaction between residues far apart in the sequence may affect formation of the secondary structure strongly. Incorporation of the long-range interactions becomes one of major problems for the protein secondary structure prediction in the next step.

Acknowledgement The authors would like to thank Professor James A. Cuff and Geoffrey J. Barton for providing the CB513 data set. This research was supported in part by the U.S. National Institutes of Health under grants R01 GM34766-17S1, and P20 GM065762-01A1, and the U.S. National Science Foundation under grants ECS-0196569, and ECS-0334813. This work was also supported by the Georgia Cancer Coalition and used computer hardware supplied by the Georgia Research Alliance.

References

1. Altschul SF, Madden TL, Schaffer AA, Zhang J, Zhang Z, Miller W, Lipman D (1997) Gapped BLAST and PSI-BLAST: a new generation of protein database search program. *Nucleic Acids Res* 25:3389–3402

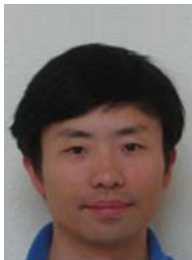
2. Arjunan SV (2003) protein secondary structure prediction from amino acid sequences using a neural network classifier based on the Dempster-Shafer theory. University Technology Malaysia, Masters Thesis
3. Butenhof D (1997) Programming with POSIX threads, Addison-Wesley Professional Computing Series
4. Chandra R, Dagum L, Kohr D, Maydan D, McDonald J, Menon R (2000) Parallel programming in OpenMP. Morgan Kaufmann Publishers
5. Cuff J, Barton G (1999) Evaluation and improvement of multiple sequence methods for protein secondary structure prediction. *Protein* 34:508–519
6. Denoeus T (2000) A neural network classifier based on Dempster-Shafer theory. *IEEE Trans Syst Man Cybern A* 30(2):131–150
7. Eggers S, Emer J, Levy H, Lo J, Stamm R, Tullsen D (1997) Simultaneous multithreading: a platform for next-generation processors, *IEEE Micro*, pp 12–18
8. Fedorova N, Terekhoff SA (1999) Parallel MPI implementation of training algorithms for medium-size feedforward neural networks. In: International joint conference on neural network, vol 4, June 1996, pp 2378–2379
9. Fathy S, Syiam M (1996) A parallel design and implementation for backpropagation neural network using MIMD architecture. In: IEEE international conference on neural networks, vol 2, June 1996, pp 1361–1366
10. Hua S, Sun Z (2001) A novel method of protein secondary structure prediction based on an improved support vector machines approach. *J Mol Biol* 308:397–407
11. Henikoff S, Henikoff JG (1992) Amino acid substitution matrices from protein blocks. *Proc Natl Acad Sci USA* 89:10915–10919
12. Karp G (2002) Cell and molecular biology, 3rd edn. pp 55–57
13. Kim H, Park H (2003) Protein secondary structure prediction based on an improved support vector machine approach. *Protein Eng* 16(8):553–560
14. Johansson C, Lansner A (2001) A parallel implementation of a Bayesian neural network with hypercolumns. Technical Report, TRITA-NA-P0121, SANS-Nada-KTH
15. Liu X, Wilcox GL (April 1993) Benchmarking of the CM-5 and the Cray machines with a very large backpropagation neural network. Technical Report 93/38, University of Minnesota Supercomputer Institute, Minneapolis
16. Marr D, Binns F, Hill D, Hinton G, Koufaty D, Miller J, Upton M (2002) Hyper-Threading Technology architecture and microarchitecture. *Intel Technol J*
17. Qian N, Sejnowski T (1988) Predicting the secondary structure of globular proteins using neural network models. *J Mol Biol* 202(4):865–884
18. Rost B, Sander C (1994) Combining evolutionary information and neural networks to predict protein secondary structure. *Protein* 19:55–72
19. Rost B, Sander C (1992) Exercising multi-layered networks on protein secondary structure. *Int J Neural Syst* 3:209–220
20. Rost B, Sander C (1993) Prediction of secondary structure at better than 70% accuracy. *J Mol Biol* 232:584–599
21. Tian X, Bik A, Girkar M, Grey P, Saito H, Su E (2002) Intel OpenMP C++/Fortran compiler for Hyper-Threading Technology: implementation and performance. *Intel Technol J* 6(1)
22. Silva F, Alemida L (1990) Speeding up backpropagation, advanced neural computers. North-Holland, Amsterdam, pp 151–158
23. Suresh S, Omkar SN, Mani V (2003) Parallel implementation of memory neuron network for identification of dynamical system. *Adv Vibr Eng* 2(2)
24. Thulasiram R, Rahman RM, Thulasiraman P (2003) Neural Network Training Algorithms on Parallel Architectures for Finance Applications. In: ICPP workshops, 2003, pp 236–243
25. Tanomaru J, Omichi S, Azuma A (1995) General purpose MIMD computers and neural networks: three case studies. In: IEEE international conference on systems, man and cybernetics, 1995, pp 4587–4597
26. Weishäupl T, Schikuta E (2003) Parallelization of cellular neural networks for image processing on cluster architectures. In: International conference on parallel processing workshops. Kaohsiung, Taiwan, October 06–09, 2003



Wei Zhong received his B.S. degree and Ph.D. degree in computer science from Georgia State University in 2001 and 2006. Currently, he is the assistant professor in the division of Math and Computer Science at University of South Carolina Upstate. He received the Outstanding Ph.D. Research Award from Georgia State University for his works on bioinformatics. He has served as a reviewer for many conferences and journal papers. He is supported by GSU Molecular Basis of Disease Program Fellowship. His research is also supported by several NIH and NSF grants. His main research interests include bioinformatics, machine learning algorithms and data mining.



Gulsah Altun was born in Canakkale, Turkey. She received her B.S. degree in Electronics and Telecommunication engineering from Kocaeli University, Turkey in 1999 and her M.S. degree in Computer Science from Georgia State University, USA in 2003. She is currently pursuing the Ph.D. degree in the Department of Computer Science at Georgia State University under the supervision of Dr. Robert Harrison. She is the president of the Student Chapter of ACM (Association for Computing Machinery) at Georgia State University and she has served as a reviewer for many conferences and journal papers. Her research interests include bioinformatics, data mining, and parallel computing.



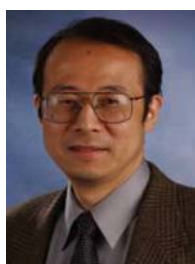
Xinmin Tian is a Principal Engineer at Intel Corporation and manages an Intel research and development group working on exploiting thread-level parallelism in Intel® C++ and Fortran compilers for Intel® IA-32, Intel64 and Itanium® multi-core architectures. Dr. Tian earned a Ph.D. in computer science from Tsinghua University in 1993, and was a post-doctoral researcher at McGill University in 1994 and 1995. Dr. Tian has over 30 refereed technical publications on compiler optimizations, parallel computing, and multithreaded architectures. He is the co-author of The Software Optimization Cookbook (Second Edition) at Intel Press published in 2006, and a main contributor for Multi-Core Programming book at Intel Press published in 2006. Dr. Tian has 20 patents pending in the areas of compiler optimizations, parallelization, and multi-core architectures. Dr. Tian served on program committee for research conferences, a referee for technical journals and conferences, and an OpenMP ARB committee member for Intel.



Robert Harrison received the B.S. degree in Biophysics from Pennsylvania State University, University Park PA in 1979 and the Ph.D. degree in Molecular Biochemistry and Biophysics from Yale University, New Haven CT in 1985. He is currently an Associate Professor in the department of computer science at Georgia State University in Atlanta GA. An active scientist, he has published over 95 papers in journals and referred proceedings on a wide range of computational issues in computational biology, structural biology and bioinformatics. He is a Georgia Cancer Coalition Distinguished Scholar and his research has been supported by the National Institutes of Health and the Georgia Cancer Coalition. His current research interests include computational approaches to the prediction and design of molecular structure, machine learning, and the development of grammar-based models for systems and structural biology.



Phang C. Tai received the Ph.D. degree in microbiology from the University of California, Davis. He has done postdoctoral work at Harvard Medical School, Boston, MA, and is currently a Regents' Professor and Chair of the Department of Biology, Georgia State University, Atlanta. His research interest is in molecular biology and microbial physiology. His current research focuses on the mechanism of protein secretion across bacterial membranes, with emphasis on the structure and function of SecA protein.



Yi Pan is the chair and a professor in the Department of Computer Science and a professor in the Department of Computer Information Systems at Georgia State University. Dr. Pan received his B.Eng. and M.Eng. degrees in computer engineering from Tsinghua University, China, in 1982 and 1984, respectively, and his Ph.D. degree in computer science from the University of Pittsburgh, USA, in 1991. Dr. Pan's research interests include parallel and distributed computing, optical networks, wireless networks, and bioinformatics. Dr. Pan has published more than 100 journal papers with 30 papers published in various IEEE journals. In addition, he has published over 100 papers in refereed conferences (including IPDPS, ICPP, ICDCS, INFOCOM, and GLOBECOM). He has also co-authored/co-edited 30 books (including proceedings) and contributed several book chapters. His pioneer work on computing using reconfigurable optical buses has inspired extensive subsequent work by many researchers, and his research results have been cited by more than 100 researchers worldwide in books, theses, journal and conference papers. He

is a co-inventor of three U.S. patents (pending) and 5 provisional patents, and has received many awards from agencies such as NSF, AFOSR, JSPS, IISF and Mellon Foundation. His recent research has been supported by NSF, NIH, NSFC, AFOSR, AFRL, JSPS, IISF and the states of Georgia and Ohio. He has served as a reviewer/panelist for many research foundations/agencies such as the U.S. National Science Foundation, the Natural Sciences and Engineering Research Council of Canada, the Australian Research Council, and the Hong Kong Research Grants Council. Dr. Pan has served as an editor-in-chief or editorial board member for 15 journals including 5 IEEE Transactions and a guest editor for 10 special issues for 9 journals including 2 IEEE Transactions. He has organized several international conferences and workshops and has also served as a program committee member for several major international conferences such as INFOCOM, GLOBECOM, ICC, IPDPS, and ICPP. Dr. Pan has delivered over 10 keynote speeches at many international conferences. Dr. Pan is an IEEE Distinguished Speaker (2000–2002), a Yamacraw Distinguished Speaker (2002), a Shell Oil Colloquium Speaker (2002), and a senior member of IEEE. He is listed in Men of Achievement, Who's Who in Midwest, Who's Who in America, Who's Who in American Education, Who's Who in Computational Science and Engineering, and Who's Who of Asian Americans.